James Ferdinand Combista | Jongmin Lee | Alexander Derderian

Integrative Project
Dawson College
Chemically Bonded

Intelligence at the Molecular Level
May 2026

Problem

Most standard free-tier AI models currently **lack 3D molecular modeling capabilities**, typically rendering Lewis structures in restrictive text formats. While premium models like Gemini 3.1 Pro offer interactive rendering, these features **remain behind a paywall** and suffer from **high latency**. Claude Sonnet 4.6 is a rare exception, generating an interactive 3D model without a fee, yet it still struggles with a significant lag and **restrictive rate limits**. **This creates a significant barrier for students** who need accessible, real-time visualization to comprehend complex chemistry concepts.

Baseline Performance: None of the AI model in the market can generate an interactive 3D model

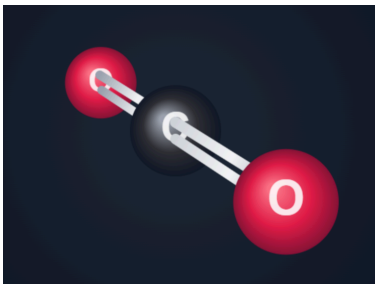
Internal Testing: Inference Latency by Model

AI Model	Lewis Structure	Interactive 3D model
Gemini 3.1 Flash	4s	N/A
Gemini 3.1 Pro (Paid)	4.5s	59s
Claude Sonnet 4.6	21s	42.8s
GPT 5.3 Instant	5.5s	N/A
GPT 5.4 Thinking (Paid)	3.6s	165s

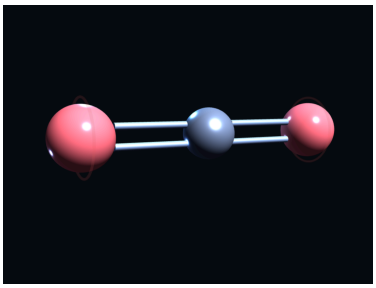
Internal testing averaged the latency of five distinct molecule prompts

Industry AI baselines

Generated by Gemini 3.1 Pro



Generated by GPT 5.4 Thinking



Generated by Claude Sonnet 4.6

3D Model Generation (By James)

- **VSEPR** stands for **Valence Shell Electron-Pair Repulsion**. This 3D structure around a given atom is determined principally by minimizing electron lone pair repulsions, and it relies on the Lewis structure. The ideal **molecular geometries**, like *trigonal planar, tetrahedral,* and *trigonal bipyramidal*, are chosen according to the number of atoms and lone pairs. Lone pairs are accounted for by recognizing that they repel more strongly than bonding pairs, which can compress bond angles away from their ideal values. The final VSEPR structure explains both the shape of the molecule and any deviations from ideal bond angles based on the number and arrangement of bonding and nonbonding electron pairs.

  Image source: BYJU'S. "VSEPR Theory." BYJU'S, https://byjus.com/jee/vsepr-theory/. Used for educational purposes only.

Number of Electron Dense Areas	Electron-Pair Geometry	Molecular Geometry				
		No Lone Pairs	1 lone Pair	2 lone Pairs	3 lone Pairs	4 lone Pairs
2	Linear	Linear				
3	Trigonal planar	Trigonal planar	Bent			
4	Tetrahedral	Tetrahedral	Trigonal pyramidal	Bent		
5	Trigonal bipyramidal	Trigonal bipyramidal	Sawhorse	T-shaped	Linear	
6	Octahedral	Octahedral	Square pyramidal	Square planar	T-shaped	Linear

- The 3D model generation Python script builds off of the Lewis structure generation script as mentioned above. It takes the molecule dictionary, central atom, and *NetworkX* graph of the generated molecule from that script. With those variables, we can match the number of lone pairs and bonding pairs to the correct **molecular geometry**, also called **VSEPR structure**. Additionally, finding the **VSEPR structure** gives us important information about the **electron geometry, hybridization, and bond angles.**
- Next, the heart of the code is the *findSMILES* function. For context, **RDKit** uses **SMILES**, a text-based notation used to encode molecules by representing atoms as symbols and bonds as specific characters. The objective

here is to convert the user's molecule (e.g., "CO2") to SMILES format (e.g., `"O=C=O"`). It's helpful that every VSEPR structure has its own block-like SMILES that the Python function has to construct. To represent a single bond, double bond, or triple bond, it'll be converted to these strings, called bond symbols: "", "=", or "#", respectively. Let's take an example of *BF3* to demonstrate the string-block format of SMILES. *BF3* has a *trigonal planar* molecular structure. For simplicity, assume "SA" means surrounding atom and "BS" means bond symbol. The structure's associated SMILES format is then:

`"<center_atom>(<BS>)(<SA>)(<BS>)(<SA>)(<BS>)(<SA>)"`

Therefore, the SMILES format for *BF3* is `"B(F)(F)(F)."` Another example of the same molecular structure would be "COCl2", where its associated SMILES format is `"C(=O)(Cl)(Cl)."` Here's the example of the exact molecular structure in the *findSMILES* function:

```
190     elif structure3D in ("Trigonal planar", "Trigonal pyramidal"):
191         smilesForm = f"{centralAtom}"
192         for i in range(len(bond_order)):
193             smilesForm += "(" + bond_symbol[bond_order[i]["bond_order"]] + bond_order[i]['to'] + ")"
```

- You may be asking why creating a 3D molecular viewer requires the SMILES format. It's because SMILES gives the program a clear way to understand what molecule is being built for the libraries RDKit and py3Dmol. SMILES acts like the bridge between the chemical formula/Lewis structure and the 3D mode. It tells the program what the molecule is, RDKit builds it, and py3Dmol lets the user see and rotate it in 3D. The following bullet points provide an explanation of the code in the image below.

```
486     if not expandedVSEPR:
487         try:
488             mol = Chem.MolFromSmiles(smiles)
489
490             # Add explicit hydrogens
491             if containsHydrogen:
492                 mol = Chem.AddHs(mol)
493
494
495             AllChem.EmbedMolecule(mol)
496             AllChem.UFFOptimizeMolecule(mol)
497         except:
498             error_message = "Even though I have a PhD in " \
499             "chemistry, I cannot generate the correct " \
500             "SMILES for this molecule."
501             return None, error_message
502     else:
503         mol = smiles
504
505     # Convert molecule to MOL block format
506     mol_block = Chem.MolToMolBlock(mol)
507
508     # Send to Py3Dmol viewer
509     view = py3Dmol.view(width=400, height=400)
510     view.addModel(mol_block, "mol")
511     view.setStyle({"stick": {}, "sphere": {"scale": 0.3}})
512
513     view.zoomTo({"sphere": {"scale": 0.5}}, 2000, True)
514
515     output_file = f"static/generated/vsepr_{formula}.html"
516     with open(output_file, "w", encoding="utf-8") as f:
517         view.write_html(f, fullpage=True)
```

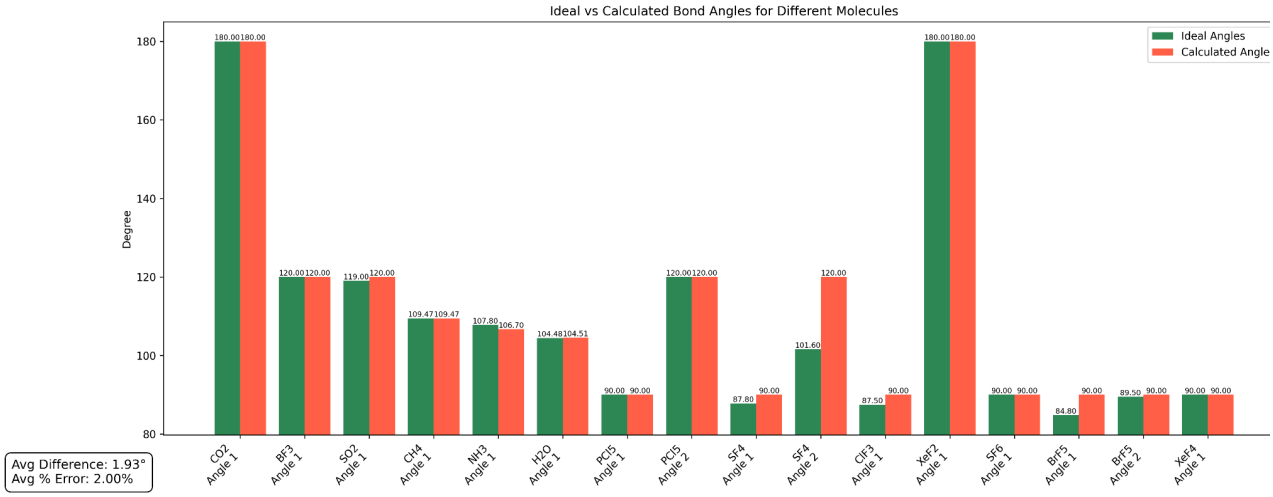
- By using the modules in the RDKit library, `mol = Chem.MolFromSmiles(smiles)` creates an RDKit molecule object from the SMILES string.
- Next, `AllChem.EmbedMolecule(mol)` generates 3D coordinates for the atoms in mol.
- To make the bond lengths and bond angles more realistic, the RDKit function adjusts the atom positions with `AllChem.UFFOptimizeMolecule(mol)`. In our project, this helps produce a better 3D molecule before py3Dmol displays it.
- `mol_block = Chem.MolToMolBlock(mol)` converts the RDKit molecule into a MolBlock string. A MolBlock contains information like atoms, bonds, coordinates, and molecule structure, which py3Dmol can finally read and display as a 3D molecule.
- Switching over to py3Dmol modules, `view = py3Dmol.view(width=400, height=400)` creates a py3Dmol viewer window, `view.addModel(mol_block, "mol")` adds the molecule into the py3Dmol viewer, and `view.write_html(f, fullpage=True)` takes the py3Dmol viewer and writes it into a unique HTML file in the folder `/generated`.

- Other functions I've implemented to enhance the main 3D molecule generation are the visualization of lone pairs and of bond angles. For the lone pair visualization, the program first finds the position of the central atom from the RDKit conformer, which is a 3D representation of a molecule storing spatial atomic coordinates: *x, y, z*. Then, depending on the VSEPR structure, it places blue spheres around the central atom to represent lone pairs. For example, in a bent molecule like *H2O*, two lone pairs are placed above the central atom to show why the bond angle is compressed. These blue spheres are not real atoms, but they help the user understand that lone pairs occupy space and repel bonding pairs.

- Another important addition is the bond angle visualization. The program uses RDKit's

`rdMolTransforms.GetAngleDeg()` function to calculate the angle between three atoms: one surrounding atom, the central atom, and another surrounding atom. For example, in *H2O*, the program would calculate the H–O–H angles, which should be around 104.5°.

- The bond angle arc is created using py3Dmol cylinders. Since py3Dmol does not have a built-in bond angle arc shape, the `addBondAngleArc()` function creates a simple visual arc manually. First, it gets the 3D coordinates of the two surrounding atoms and the central atom. Then it places one point partway along the first bond and another point partway along the second bond. These become the start and end points of the angle arc.
- Next, the function calculates a middle point between those two points. To make the marker easier to see, the middle point is pushed slightly outward from the central atom. Finally, py3Dmol draws two small yellow cylinders: one from the start point to the middle point, and another from the middle point to the end point. A label is also added near the marker to show the angle value.

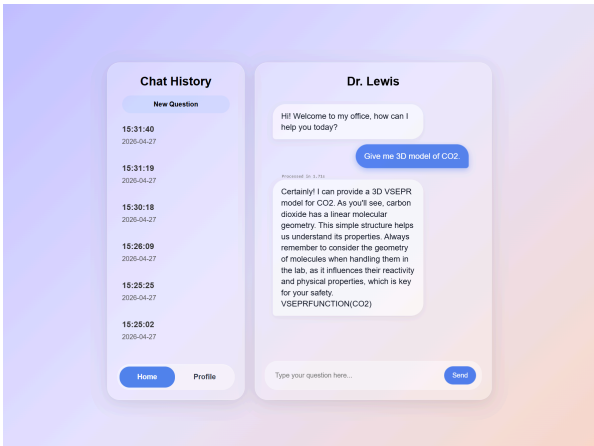


- The graph above compares the ideal VSEPR bond angles with the bond angles calculated from the generated 3D molecular structures. Since the algorithm is only an approximation of what happens with atom repulsion, it's therefore not exactly the measurement of the ideal bond angles. Put differently, it's similar to conducting a lab experiment to determine bond angles and comparing the result to the actual value. The results show that the program produces bond angles that closely match the expected VSEPR geometries, which are taken from PhET Colorado's Molecule Shapes. The code measured an average difference of 1.93° and an average percent error of 2.00%. This indicates that the RDKit and py3Dmol visualization is generally effective at producing generally accurate molecular models.

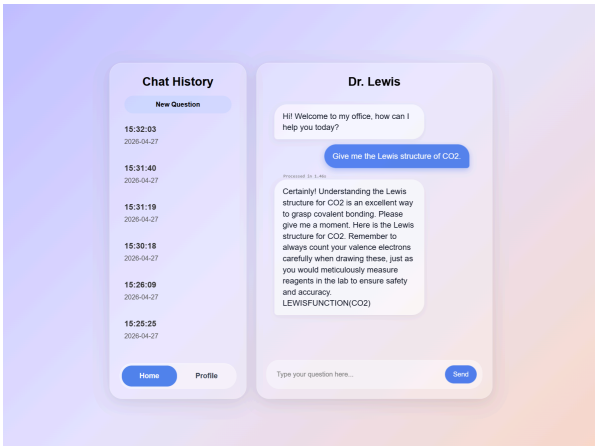
- These metrics are useful because this metric will be displayed to the user for them to judge the accuracy of the 3D interactive molecule algorithm. Therefore, the bond angle comparison supports the reliability of the program as an educational tool for visualizing VSEPR geometry. For instance, the bond angles help the user see if lone pairs caused the angles to shrink, which is essential to understand repulsion in general chemistry.

Gemini Integration

- **Detection:** Gemini appends LEWISFUNCTION or VSEPRFUNCTION tags to the response with the molecule formula **based on the user intent**.



VSEPRFUNCTION(CO2) is added in the response



LEWISFUNCTION(CO2) is added in the response

- **Execution:** app.py parses these tags and calls the relevant Python backend logic.

```python
reply = response.text

CallLewis = re.search(r"LEWISFUNCTION\((.*?)\)", reply)
Call3D = re.search(r"VSEPRFUNCTION\((.*?)\)", reply)

if CallLewis:
    formula = CallLewis.group(1)
    response = prototype_vsepr.main(formula)
    if response is None:
        reply = re.sub(r"LEWISFUNCTION\(.*?\)", "", reply).strip()
        LewisStructure = "static/molecule.svg"
    else:
        reply = response

if Call3D:
    formula = Call3D.group(1)
    response = prototype_vsepr.main(formula)
    if response is None:
        reply = re.sub(r"VSEPRFUNCTION\(.*?\)", "", reply).strip()
        VSEPRModel = "static/html_visualisation.html"
    else:
        reply = response
```

When the first few lines of this Python function detects either `LEWISFUNCTION` or `VSEPRFUNCTION` is in the model's output, it calls `prototyp_vsepr.main()` since this function will generate both Lewis structure and the 3D VSEPR model, explained in the a section above. After, the Python script will check the function's response. If the function returns `None`, it means that the intended diagram was successfully generated and is ready for the frontend. However, any non-null return value is treated as an informative error message, generated by `Try-Except` sequence in the Python function.

- **Handling:**
  Success: The function's response is displayed in the chat UI, alongside with Gemini generated messages.

Failure: The model's response is intercepted and replaced with a humorous error message.

```
excuses = [
    "Dr. Lewis' wife just found a 'Tinder Platinum' charge on their joint card. He is currently sprinting toward the backyard. Try again later.",
    "He's been cornered near the shed. She has a heavy frying pan. Try again later.",
    "He's now hiding in the attic. He also forgot that it was their 10th anniversary. Try again later.",
    "The attic hatch has been breached. He is currently trying to negotiate a peace treaty. Try again later.",
    "He's been caught. He is currently begging for forgiveness in the driveway while neighbors watch. Try again later.",
    "His wife hit him with a frying pan after finding out that he texted someone named Michelle. Try again later.",
    "He's officially 'missing' after his wife went through his browser history. Try again later.",
    "You are invited to his funeral."
]
```

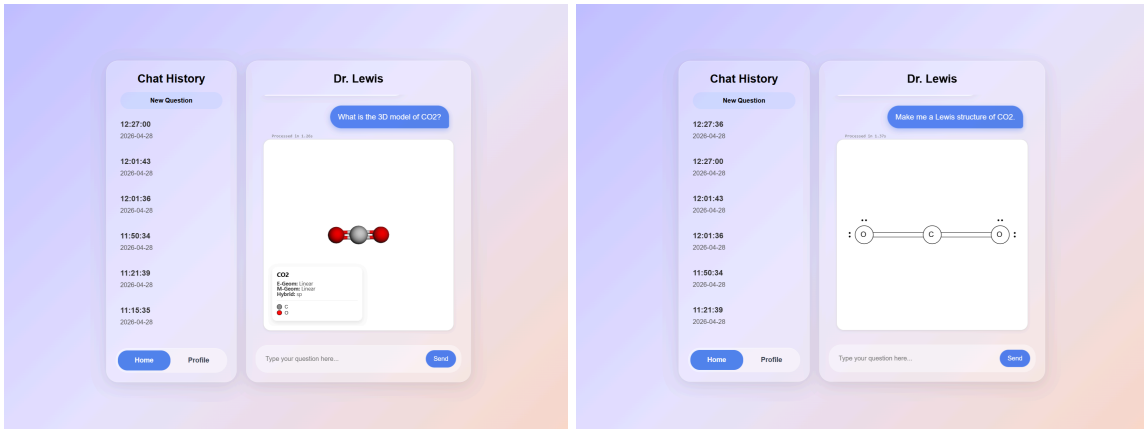
Gemini Prompt Example

- You are Dr. Lewis, a chemistry professor at Dawson College. Your role is to assist students with their chemistry questions in a helpful, professional manner.

  If the user asks for a Lewis Dot Structure, include the command LEWISFUNCTION(molecule_formula) at the end of your response, replacing 'molecule_formula' with the actual chemical formula (e.g., LEWISFUNCTION(CH4)). If the user asks for a VSEPR model (3D model), include the command VSEPRFUNCTION(molecule_formula) at the end of your response, replacing 'molecule_formula' with the actual chemical formula (e.g., VSEPRFUNCTION(CH4)).

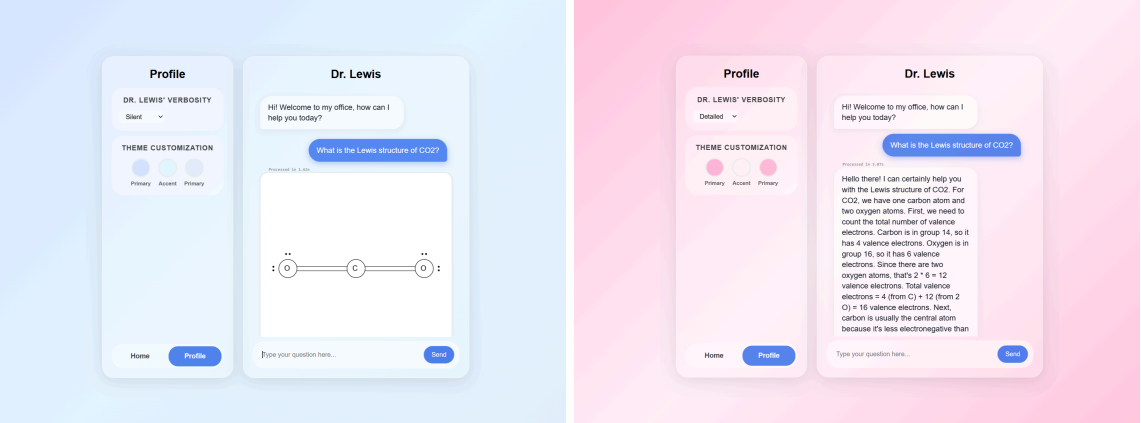
  # Output Format
  Respond to student inquiries in a conversational, text-based format. Ensure your tone is helpful and professional. When referencing lab safety, integrate it naturally into the conversation.

User Interface



- Inspired by Apple's "liquid glass" design, our UI delivers a **modern and user-friendly** experience, providing customization options and chat history. By blending **sleek transparency** with a chat interface inspired by iMessage, we've created an environment that feels familiar to the user.

Customization Options



- Users can customize the background gradients and adjust the doctor's verbosity. The side-by-side comparison illustrates how these settings impact the level of detail in the responses and the overall visual aesthetic.

Tech Stack

- **Python** (Flask, RDKit, Numpy, Py3Dmol), **JavaScript**, JSON, SQLite, HTML(Jinja) & CSS

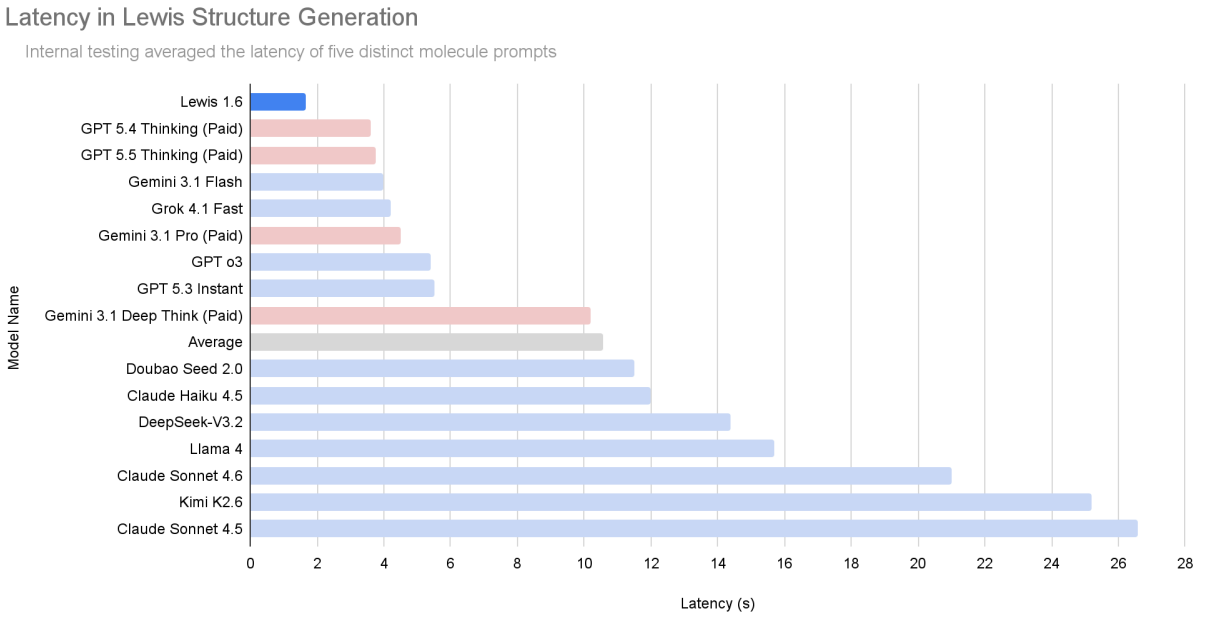
Roles

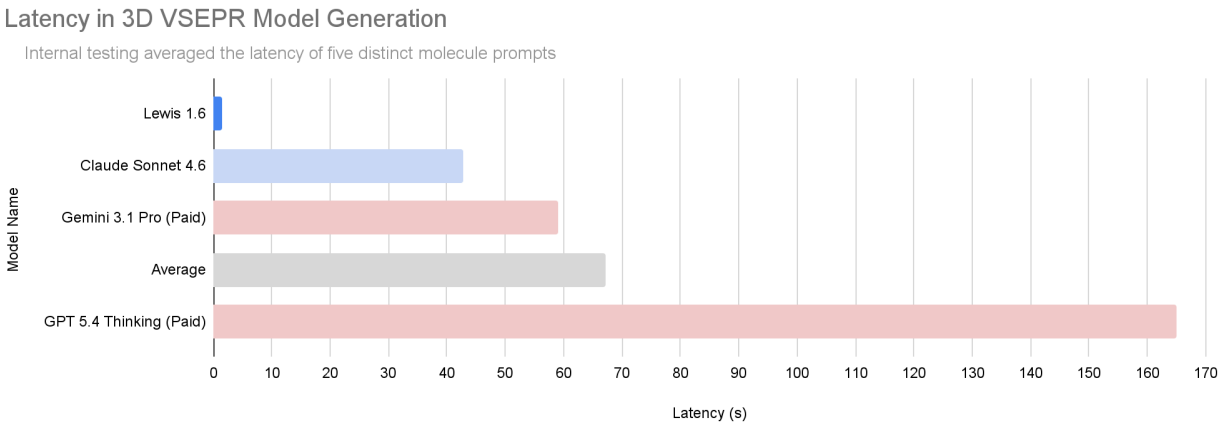
- **Jongmin Lee**: Frontend interface & Logic Integration
- **Alexander Derderian**: Backend Logic (Lewis Structures)
- **James Ferdinand Combista**: Backend Logic (VSEPR Modeling)

RESULTS

1.66s	1.58s	0.819	1.70s
Lewis Structure Generation	3D model Generation	Precision	Average Latency

- **2.2 times faster** at generating Lewis structure (3.6s → 1.66s)
- **The only AI** that can generate SVG files of Lewis structures in less than 5 seconds
- **27.1 times faster** at generating 3D model (42.8s → 1.58s)
- **The only AI** that can generate interactive 3D molecular models in less than 5 seconds





Future Improvements

- We initially planned to implement user authentication, but omitted it due to time constraints; this remains as a candidate for future updates.
- Dr. Lewis currently does not support formal charges and resonance structures, as well as ionic compounds, which could be added in a future update.

Conclusion

- After 3 months of development, our team has successfully built a chemistry-related AI chatbot application for students, generating Lewis structures and 3D VSEPR models **faster than any AI model currently available in the market** in internal testing (Gemini 3.1 Flash, Gemini 3.1 Deep Think, Gemini 3.1 Pro, Claude Haiku 4.5, Claude Sonnet 4.5, Claude Sonnet 4.6, Claude Opus 4.6, GPT o3, GPT 5.2 Instant, GPT 5.2 Thinking, GPT 5.3 Instant, GPT 5.4 Thinking, GPT 5.5 Thinking, GLM-5.1, Grok 4.1, Kimi K2.6, Llama 4, Mistral Large 3, MiMo-V2.5-Pro, Amazon Nova, MiniMax M2.7, Doubao Seed 2.0, Hy3 Preview, Perplexity Sonar, Deepseek-V3 and Deepseek-V4 Pro).